

(I,PSF,NSR) deconvolves image I using Wiener filter algorithm, returning deblurred image J. The following commands generate a deblurred image using NSR of zero:

```
>>estimated_nsr=0;
>>wnc2=deconvwnr(blurred_noise,PSF,
    estimated_nsr);
>>figure,imshow(wnc2)
```

Fig. 4 shows the image generated by imshow function.

5. The following commands generate a deblurred image using NSR calculated from the image:

```
>>estimated_nsr=noise_var/var(I(:));
>>wnc3=deconvwnr(blurred_noise,PSF,
    estimated_nsr);
>>figure,imshow(wnc3)
```

Fig. 5 shows the restored image from the blurred and noisy image using estimated NSR generated from the image.

Deblurring with a regularised filter

A regularised filter can be used effectively when limited information is known about the additive noise.

$J = \text{deconvreg}(I, \text{PSF})$ deconvolves image I using regularised filter algorithm and returns deblurred image J. The assumption is that image I was created by convolving a true image with a point-spread function and possibly by adding noise. The algorithm is a constrained optimum in the sense of least square error between the estimated and true images under requirement of preserving image smoothness.

Image I can be an N-dimensional array.

Variations of deconvreg function are given below:

- $J = \text{deconvreg}(I, \text{PSF}, \text{NOISE-POWER})$

where NOISEPOWER is the additive noise power. The default value is 0.

- $J = \text{deconvreg}(I, \text{PSF}, \text{NOISE-POWER}, \text{LRANGE})$

where LRANGE is a vector specifying range where the search for the optimal solution is performed.

The algorithm finds an optimal Lagrange multiplier LAGRA within LRANGE range. If LRANGE is a scalar, the algorithm assumes that LAGRA is given and equal to LRANGE; the NP value is then ignored. The default



Fig. 3: Image generated after addition of Gaussian white noise



Fig. 4: Deblurred image using NSR of zero



Fig. 5: Restored image from the blurred and noisy image using estimated NSR generated from the image



Fig. 6: Image read in MATLAB



Fig. 7: The blurry and noisy image

function. Using these arguments you can specify the noise power value, the range over which deconvreg should iterate as it converges on the optimal solution, and the regularisation operator to constrain the deconvolution.

Image I can be of class uint8, uint16, int16, single or double. Other inputs have to be of class double. Image J has the same class as Image I.

The following example simulates a blurred image by convolving a Gaussian filter PSF with an image (using imfilter):

1. Read an image in MATLAB:

```
>> I=imread('image_2.tif');
>>figure,imshow(I)
```

Fig. 6 shows the image.

2. Create the PSF, blur the image and add noise to it:

```
>> PSF = fspecial('gaussian',11,5);
>> Blurred = imfilter(I,PSF,'conv');
>> V=0.03;
>> Blurred_Noise=imnoise(Blurred,
    'gaussian',0,V);
>>figure,imshow(Blurred_Noise)
```

Note that additive noise in the image is simulated by adding Gaussian noise of variance V to the blurred image (using imnoise). Fig. 7 shows the blurry and the noisy image.

3. The image is deblurred using deconvreg function, specifying the

range is between [1e-9 and 1e9].

- $J = \text{deconvreg}(I, \text{PSF}, \text{NOISE-POWER}, \text{LRANGE}, \text{REGOP})$

where REGOP is the regularisation operator to constrain deconvolution. The default regularisation operator is Laplacian operator, to retain the image smoothness. REGOP array dimensions must not exceed image dimensions; any non-singleton dimensions must correspond to the non-singleton dimensions of PSF.

$[J, \text{LAGRA}] = \text{deconvreg}(I, \text{PSF}, \dots)$ outputs the value of Lagrange multiplier LAGRA in addition to the restored image J.

In a nutshell, there are optional arguments supported by deconvreg